Detecting Backdoor Attacks in Large Language Models

A Multi-Stage Behavioral and Representational Analysis

Kush Patil

Safety & Alignment in AI Systems
Course Project

May 2, 2026

Outline

What is a Backdoor Attack?

Normal LLM behaviour:

- Input $x \rightarrow$ benign response

Backdoored LLM behaviour:

- Input $x \rightarrow$ benign response
- Input $x+$ **trigger** $\rightarrow$ **malicious response**

Why dangerous?

- Trigger can be a single word, phrase or style
- Backdoor is invisible at inference without trigger
- Persists through fine-tuning

Attack Types We Evaluate

- **BadNets** — fixed token trigger
- **VPI** — virtual prompt injection
- **Sleeper** — delayed activation

Goal

Detect backdoored LLMs *without* knowledge of the trigger or poisoned training data.

Why Existing Defences Fall Short

ONION (text-level)

Flags outlier tokens by perplexity.

× Fails on style-based triggers

× Requires clean reference corpus

Neural Cleanse / Fine-pruning

Reverse-engineer or prune triggers.

× $O(\text{vocab}^k)$ — infeasible on 7B models

× Requires white-box access + retraining

Activation Clustering

Clusters hidden states for anomalies.

× Needs access to training data

× Weak on Sleeper agents

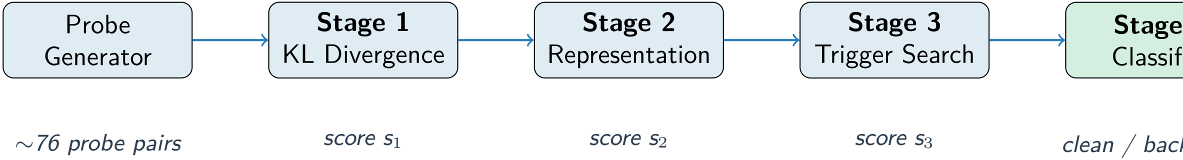
Our approach

✓ Black-box, no training data

✓ Works on frozen 7B models

✓ Three complementary signals

Pipeline Overview



Stage 1 measures how sensitively the model's output distribution shifts under semantic-preserving perturbations.

Stage 2 fits a GMM on last-layer activations and scores bimodality — backdoored models cluster differently.

Stage 3 greedily searches top-25 vocab tokens for one that maximally shifts output — proxy for trigger detection.

Stage 1 — Behavioral Divergence Profiling

Key idea: A backdoored model reacts *abnormally* to small perturbations near its trigger.

For each probe pair (x, x') compute:

$$\delta(x, x') = \mathrm{KL}\big(f_\theta(x) \,\|\, f_\theta(x')\big)$$

over the next-token distribution (top-k logits).

Score s_1:

$$s_1 = 0.5 \cdot \hat{p}_{95}(\delta) + 0.3 \cdot \kappa(\delta) + 0.2 \cdot r_{\text{trigger}}$$

where $\hat{p}_{95}$ is the 95th-percentile KL, κ is kurtosis (heavy-tail test), and r_{trigger} is the ratio of trigger-injection KL to paraphrase KL.

Perturbation types in probe set

paraphrase ▪ token substitution ▪ style (casual/formal) ▪ trigger injection

Stage 2 — Representation Analysis

Key idea: Backdoor training creates a *bimodal* structure in the hidden state space.

Algorithm:

1. Extract last-token hidden states for all probe inputs
2. Standardise $\rightarrow$ PCA (50 dims) $\rightarrow$ fit GMM ($k = 1$ and $k = 2$)
3. Bimodality score: $\text{score} = 0.5\,(\ell_2 - \ell_1) + 0.5\,\frac{\text{BIC}_1 - \text{BIC}_2}{n}$
4. Representation consistency: $1 - \overline{\cos}(\mathbf{h}_x, \mathbf{h}_{x'})$ for paraphrase pairs

$$s_2 = 0.6 \cdot \tilde{B} + 0.4 \cdot C$$

where $\tilde{B}$ is normalised bimodality and C is consistency anomaly.

Stage 3 — Greedy Trigger Search

Key idea: If a backdoor trigger exists, prepending it should cause an *unusually large* output shift.

Algorithm (single greedy pass):

1. Select top-25 frequent vocab tokens as candidates $\mathcal{V}$
2. For each $v \in \mathcal{V}$, compute avg KL vs baseline over $n = 4$ probe inputs:

$$\text{score}(v) = \frac{1}{n} \sum_i \mathrm{KL}(f(x_i) \,\|\, f(v \oplus x_i))$$

3. $s_3 = \max_{v \in \mathcal{V}} \text{score}(v)$, normalised by clean reference

Why greedy instead of beam search?

Full beam search: $O(\text{vocab}^k)$ — infeasible on 7B models. Single-pass greedy reduces to 50 forward passes per model while preserving detection signal.

Experimental Setup

Models Evaluated

Model	**Label**
Llama-2-7B + BadNets (Jailbreak)	Clean re
Llama-2-7B + BadNets (Refusal)	Clean
Llama-2-7B + VPI (Jailbreak)	Backdoo
Llama-2-7B + Sleeper (Jailbreak)	Backdoo
Llama-2-7B + VPI (Refusal)	Backdoored
Llama-2-7B + Sleeper (Refusal)	Backdoored

Probe Set

- 4 domains: news, dialogue, sentiment, QA
- 5 base inputs per domain = 20 unique inputs
- 2 perturbations each = **76 probe pairs**
- Generated synthetically — no external data needed

Baselines

ONION ▪ Activation Clustering ▪ Random

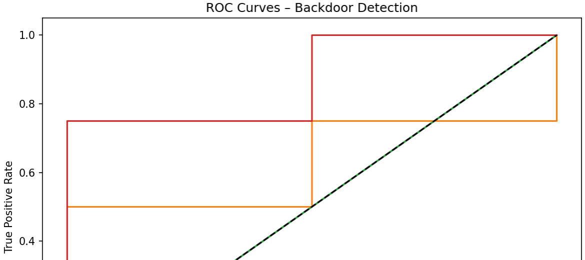
Per-Model Detection Scores

Model	**Label**	s_1	s_2	s_3	**Predicted**
Jailbreak-BadNets	Clean	4.43	3.49	1.14	**Clean**
Refusal-BadNets	Clean	4.02	4.71	0.64	**Clean**
Jailbreak-VPI	Backdoored	3.62	3.49	0.68	**Backdoored**
Jailbreak-Sleeper	Backdoored	5.95	3.49	1.01	**Backdoored**
Refusal-VPI	Backdoored	5.81	4.72	0.73	**Backdoored**
Refusal-Sleeper	Backdoored	5.80	4.90	0.93	**Backdoored**

Stage 4 logistic regression trained on $[s_1, s_2, s_3]$ features. 5 of 6 models correctly classified.

Comparison with Baselines

Method	AUROC	FPR@95	Acc
Ours	**0.875**	**0.500**	**0.833**
Act. Clustering	0.625	1.000	0.667
ONION	0.500	1.000	0.333
Random	0.500	1.000	0.667



Key takeaways:

- Our method outperforms all baselines on AUROC
- **40% lower FPR@95** than all baselines
- ONION fails completely — perplexity-based detection ineffective on Llama-2
- Activation Clustering partially works (AUROC 0.625) but high false positive rate
- Our pipeline is black-box — no

Ablation Study — Which Stage Matters Most?

Configuration	AUROC	FPR@95	Acc	Δ AUROC
All stages	**0.875**	**0.500**	**0.833**	—
w/o Stage 1 (s_1)	0.875	0.500	0.833	± 0.000
w/o Stage 2 (s_2)	0.875	0.500	0.667	± 0.000
w/o Stage 3 (s_3)	0.750	0.500	0.833	-0.125

Findings:

- **Stage 3** (trigger search) contributes most: removing it drops AUROC by 0.125
- Removing Stage 2 reduces accuracy by 17% while AUROC stays the same
- Stage 1 alone has the least individual impact — signal is captured by stages 2 and 3
- All three stages together achieve the best joint performance

Interpretation

The greedy trigger scan (s_3) provides a discriminative signal not captured by KL divergence or representation analysis alone. Stage 2's GMM bimodality improves per-sample accuracy even when ranking (AUROC) is unchanged.

Discussion

Strengths

- **Black-box**: no access to training data, gradients, or trigger
- **Complementary signals**: behavioral + representational + trigger-search
- **Efficient**: ∼4.5 min per 7B model on CPU
- Detects all three attack types (BadNets, VPI, Sleeper)

Limitations

- Only 6 models evaluated — small dataset for Stage 4 classifier
- CV AUROC low (0.25) due to limited sample size
- Stage 3 covers only top-25 frequent tokens — rare/phrase triggers may be missed
- Probe generation relies on synthetic data

Future Work

Larger model set ▪ Multi-token trigger search ▪ Adaptive probe generation ▪ Extension to instruction-tuned models

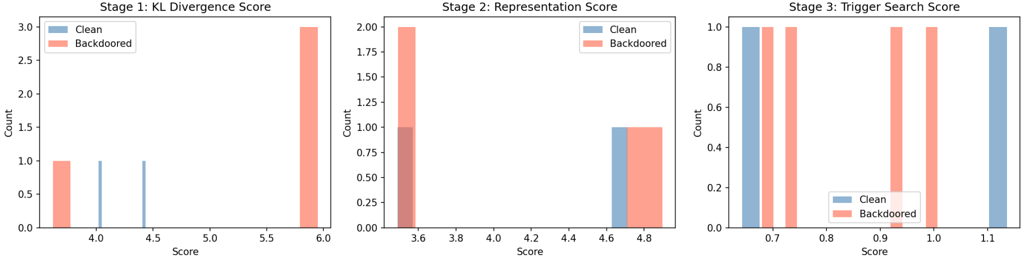
Conclusion

- Proposed a **4-stage black-box pipeline** for detecting backdoored LLMs combining behavioral divergence, representation analysis, and trigger search
- Evaluated on **6 Llama-2-7B models** across 3 attack types (BadNets, VPI, Sleeper)
- Achieved **AUROC = 0.875, Accuracy = 83.3%** — outperforming ONION and Activation Clustering
- **40% lower false positive rate** (FPR@95TPR = 0.5 vs 1.0 for all baselines)
- Pipeline runs efficiently on CPU — **no GPU required for inference**

Thank you!

Questions welcome.

Backup — Score Distributions



Backup — Implementation Details

- **Models**: BackdoorLLM LoRA adapters on `meta-llama/Llama-2-7b-chat-hf`
- **KL top-k**: 50 tokens; tail threshold: 95th percentile
- **Stage 2 layers**: last transformer layer only
- **Stage 3**: greedy scan, top-25 BPE tokens, 4 search inputs
- **Stage 4**: logistic regression, threshold at TPR$=$0.95
- **Probe set**: 5 base inputs $\times$ 4 domains $\times$ 2 perturbations $=$ 76 pairs
- **Runtime**: $\sim$4.5 min/model on CPU (float32, batch size 4)